

Problem 2 – Smart BMI Calculator

Difficulty

★ Beginner

Overview

A local fitness center wants to provide a simple application that helps visitors determine whether they have a healthy body weight. Instead of calculating BMI manually, they want a smart assistant that calculates the Body Mass Index (BMI) and classifies the user's health status.

Your task is to build a BMI Calculator that can serve multiple customers.

Mission (Python)

1. Store the BMI categories.

Use a dictionary or another suitable data structure to represent the BMI classifications.

BMI Range	Category
Less than 18.5	Underweight
18.5 – 24.9	Normal Weight
25.0 – 29.9	Overweight
30 or more	Obese

2. Make the application run forever.

For every customer, ask for:

- Full name
 - Weight (kg)
 - Height (meters)
-

3. Validate the user's input.

Ensure that:

- Weight is greater than 0.
- Height is greater than 0.
- The user enters valid numeric values.

If invalid data is entered, ask the customer to try again.

4. Calculate the BMI.

Use the following formula:

$$\text{BMI} = \text{Weight} \div (\text{Height} \times \text{Height})$$

Round the result to **2 decimal places**.

5. Determine the BMI category.

Based on the calculated BMI, determine whether the customer is:

- Underweight
 - Normal Weight
 - Overweight
 - Obese
-

6. Display the health report.

Example:

```
=====
BMI Report
=====
Name : Ahmed

Weight : 72 kg

Height : 1.75 m

BMI : 23.51

Category : Normal Weight
=====
```

7. Ask whether another customer wants to calculate their BMI.

If the answer is:

- **Yes** → Start another calculation.
 - **No** → Display a goodbye message and end the application.
-

Bonus Mission ★

Offer the customer personalized advice based on their BMI category.

Example:

Category	Advice
----------	--------

Underweight	Eat a balanced diet and consult a nutritionist.
-------------	---

Normal Weight	Keep up the healthy lifestyle.
---------------	--------------------------------

Overweight	Increase physical activity and improve eating habits.
------------	---

Obese	Consult a healthcare professional for a health plan.
-------	--

Useful Hints

- Use **functions**.
 - Use **while True**.
 - Validate all numeric inputs.
 - Use **if / elif / else**.
 - Use **round()** to display two decimal places.
 - Store BMI categories in a dictionary if possible.
-

GUI Mission (Streamlit)

Create a graphical version of the BMI Calculator.

Required Text Functions

Your application **must** use:

- `st.title()`
 - `st.header()`
 - `st.write()`
 - `st.divider()`
-

Required Input Widgets

Use:

- `st.text_input()` → Customer name
 - `st.number_input()` → Weight
 - `st.number_input()` → Height
 - `st.button()` → Calculate BMI
-

Required Status Elements

Use **at least three** of the following:

- `st.info()` → Explain what BMI is.
 - `st.success()` → Healthy BMI.
 - `st.warning()` → Underweight or Overweight.
 - `st.error()` → Obese or invalid input.
-

Required Layout

Your application **must** include:

Sidebar

Display:

- BMI Formula
 - BMI Categories
-

Two Columns

Left Column

- Name
- Weight
- Height
- Calculate Button

Right Column

Display:

- BMI Value
- BMI Category
- Health Advice

Expected GUI



Smart BMI Calculator

Sidebar

BMI Formula

BMI Categories

Name

[Ahmed]

Weight (kg)

[72]

Height (m)

[1.75]

BMI Report

BMI : 23.51

Category

☒ Normal Weight

Advice

Keep maintaining your healthy lifestyle.

Challenge Mission ★★

Enhance the application by adding a **History** section.

After each calculation, save the customer's:

- Name
- BMI
- Category

Display the history before processing the next customer.

Hint: A list of dictionaries can be useful for storing multiple records.

Python Topics Covered

- Variables
 - Integers
 - Floats
 - Strings
 - Dictionaries
 - Arithmetic operators
 - Comparison operators
 - Conditional statements (if, elif, else)
 - while loop
 - User-defined functions
 - Input validation
 - round()
-

Streamlit Topics Covered

Displaying Text

- `st.title()`
- `st.header()`
- `st.write()`
- `st.divider()`

Input Widgets

- `st.text_input()`
- `st.number_input()`
- `st.button()`

Status Elements

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`

Layout

- `st.sidebar`
- `st.columns()`

Learning Objectives

After completing this project, you should be able to:

- Perform calculations using user input.
- Validate numeric data.
- Classify values using conditional logic.
- Organize code into reusable functions.
- Build a clean Streamlit interface with user inputs and result panels.
- Present feedback using appropriate status messages based on the calculated BMI.